# Qdislib — Circuit Cutting Cheat Sheet

Run circuits bigger than your device: cut, execute the pieces, recombine.

**Qdislib** is a distributed circuit-cutting library: it splits a quantum circuit into smaller subcircuits that fit your hardware, runs them independently (in parallel, even across backends or QPUs) and recombines the results classically. It is **backend-agnostic** — the same code drives Qiskit, Qibo, CUDA-Q, PennyLane and QiliSDK — and scales with PyCOMPSs.

## WAYS TO CUT

**Gate cutting** replaces a two-qubit gate by a sum of single-qubit *measure-and-prepare* operations, splitting the circuit where that gate was. Cuts **any two-qubit gate** following its angle ($\gamma = 1 + 2|\sin\phi|$ per $rzz(\phi)$, 3 for a CZ). **Cost: $6^k$** terms for $k$ cuts.

**Wire cutting** cuts a qubit's wire (its timeline) into "measure here / re-prepare there". **Cost: $8^k$** terms for $k$ cuts, $\gamma = 4$ each (3 at a golden point, found automatically).

**Hadamard gate cutting** (`gate_cutting_hadamard`) cuts a gate with one **local ancilla** per fragment instead of mid-circuit measurement — cuts **any** gate as one cut, and is the cheaper one for a gate with several interactions (iSWAP: $\gamma$ 7 against 9).

Every cut multiplies the work **exponentially**, so use as few, and as well-placed, as possible. Preview the blow-up with `cutting_overhead(circ, cuts)` before you run.

## THE WORKFLOW

Get one **expectation value** <O> in a single call, or use the **three-step** form to control how each piece runs (cluster, QPU):

```
cut = qd.find_cut(circ, max_qubits=2, wire_cut=False)
val = qd.gate_cutting(circ, cut, 'ZZZZ',
          shots=8192)      # None = exact

subs = qd.gate_cutting_subcircuits(
    circ, cut, 'ZZZZ')  # .circuits('qiskit')
vals = [qd.expectation_value(s, shots=8192,
    backend=None)  # or a QPU, per s
        for s in subs]
val = subs.reconstruct(vals)
```

`estimate(circ, 'ZZZZ', max_qubits=n)` does find-cut + cut + reconstruct in one call.

## QUICKSTART

```
pip install "qdislib[qiskit]"   # or: uv pip install ...

from qiskit import QuantumCircuit
import Qdislib as qd

circ = QuantumCircuit(4)
circ.h(0)
circ.cz(0, 1)
circ.cz(1, 2)
circ.cz(2, 3)

cut = qd.find_cut(circ, max_qubits=2,
        wire_cut=False, seed=0)
val = qd.gate_cutting(circ, cut,
        observable='ZZZZ', shots=8192, error=True)
```

Plain `pip install qdislib` brings no SDK: pick one, `[qiskit]`, `[qibo]`, `[cudaq]`, `[pennylane]`, `[qilisdk]` or `[all]`. No install at all: **Try Qdislib**, in the browser (see Learn more).

## CORE API

| | |
|---|---|
| `find_cut(circ, ...)` | Choose where to cut (returns it). |
| `gate_cutting(circ, cut)` | <O> via gate cutting. |
| `wire_cutting(circ, cut)` | <O> via wire cutting. |
| `gate_cutting_hadamard(...)` | <O> via the ancilla cut. |
| `circuit_cutting(circ, cuts)` | Any mix of gate/Hadamard/wire cuts. |
| `..._sampling(circ, cut)` | Full output distribution. |
| `..._subcircuits(...)` | 3-step: pieces + .reconstruct(). |
| `expectation_value(sub)` | Run one subcircuit. |
| `estimate(circ, ...)` | One-call cut + run + reconstruct. |
| `cutting_overhead(circ, cuts)` | Preview circuits and gamma. |
| `shots_for(circ, cuts, eps)` | Shots for an error bound (worst case). |
| `analytical_solution(...)` | Exact <O> (to check). |
| `draw_cut(circ, cuts, path)` | Fragments in colour, cuts marked. |
| `from_qasm(qasm, ...)` | Import any OpenQASM 2. |

## HADAMARD (ANCILLA) GATE CUT

An alternative gate cut (Harrow–Lowe, *arXiv:2403.01018*): each fragment carries **one extra ancilla** running a Hadamard test — **no mid-circuit measurement**, and it cuts **any** two-qubit gate (iSWAP, …) as a single cut.

```
val = qd.gate_cutting_hadamard(
        circ, ['ISWAP_3'], 'ZZ', shots=None)
# Monte-Carlo (bounded cost for many cuts):
val = qd.gate_cutting_hadamard(
        circ, cut, 'ZZ', num_samples=5000)
# three-step, runs on any backend / QPU:
subs = qd.gate_cutting_hadamard_subcircuits(
         circ, cut, 'ZZ')
val = subs.reconstruct(
  [qd.expectation_value(s) for s in subs])
```

Gates crossing one boundary share one ancilla per side by default (`joint=True`: $\gamma = 2^{k+1}-1$ instead of $3^k$). Fragments are independent → they fan out as PyCOMPSs tasks like every other cut.

## ANY MIX OF CUTS

Cut **gates** (measure-and-prepare or Hadamard) and **wires** on the **same circuit**, in any number. With `optimized=True` each fragment runs once per setting of its own cuts, so the cost **adds** over fragments instead of multiplying over cuts.

```
cuts = {'gate': ['CZ_2'], 'hadamard': [],
        'wire': [('RY_3', 'CZ_4')]}
val = qd.circuit_cutting(circ, cuts, 'ZZZ',
        optimized=True)
qd.cutting_overhead(circ, cuts, optimized=True)
```

## SAMPLING THE DISTRIBUTION

Reconstruct the full **output distribution** `{bitstring: prob}` — to read out an image or inspect a state — instead of a single number:

```
dist = qd.gate_cutting_sampling(
         circ, cut, shots=200_000)
# three-step (parallel under PyCOMPSs):
subs = qd.gate_cutting_sampling_subcircuits(
         circ, cut)
dist = subs.reconstruct(
  [qd.sample_subcircuit(s) for s in subs])
```

`wire_cutting_sampling` too. `qubits=[0, 1]` gives a marginal; over 24 qubits the result is a `FactoredDistribution`, kept in pieces. It needs **more shots** than an expectation value.

## FIND_CUT OPTIONS

| | |
|---|---|
| `max_qubits=n` | Max qubits per subcircuit (device size). |
| `max_cuts=k` | Cap the number of cuts. |
| `gate_cut / wire_cut` | Which cut types to consider. |
| `coupling_map=` | Hardware-aware: a separating cut of the longest-routing gates (SparseCut, arXiv:2511.05492). |
| `mixed=True` | Gate, Hadamard and wire cuts priced by gamma (angle-aware, joint), within time_limit (10 s). A dict for circuit_cutting. |
| `top_n=k` | The k best candidates, ranked (not with mixed). |

# Qdislib — Circuit Cutting Cheat Sheet

Run circuits bigger than your device: cut, execute the pieces, recombine.

| | |
|---|---|
| `seed=n` | Reproducible cut selection. |

## OBSERVABLES

Per-qubit Pauli string over **I X Y Z** (qubit-0-first): `'ZZZZ'`, `'ZIZIZ'`, `'XZZY'`. A Hamiltonian is a weighted sum:

```
[(0.5, 'ZZII'), (-1.0, 'IIZZ')]
# or  {'ZZII': 0.5, 'IIZZ': -1.0}
```

`I` ignores a qubit, also on a *cut* qubit.

## BACKENDS

| | |
|---|---|
| `backend=` | 'qiskit', 'IBM_BACKEND', an object; Aer by default. |
| `subs.circuits(sdk)` | 3-step: the subcircuits in any SDK. |
| `shots=None / seed=n` | Exact / reproducible, on every cut. |
| `build_backend(name, cfg)` | A configured backend, e.g. BackendConfig(use_gpu=True) on Aer. |
| `available_backends()` | List registered backends. |
| `QdBackEnd(qpu_dict=...)` | Route circuits to the shortest queue. |
| `BaseBackend` | Your own: one method, evaluate_samples. |
| `register_backend(...)` | Make it available by name. |

## CACHING

```
cache = qd.default_circuit_cache(
          'memory', max_entries=5000)
qd.gate_cutting(circ, cut, cache=cache)
```

Keeps each fragment circuit's result, for every observable read from it (`qd.use_cache(...)`: a session; `cache_key='semantic'`: equivalent circuits too, with a warning, as two different circuits can rarely share a key). Stores: `'memory'` / `'lmdb'` / `'redis'`, its password in `$QDISLIB_REDIS_PASSWORD`.

## DISTRIBUTED WITH PYCOMPSS

Qdislib is built for HPC: every subcircuit is a PyCOMPSs `@task`, so the pieces run **in parallel across cores and nodes**. The three-step workflow fans them out automatically — the same code scales from a laptop to a supercomputer.

```
import pycompss.interactive as ipycompss
ipycompss.start()  # or: runcompss --pythonpath=$(pwd) app.py
subs = qd.gate_cutting_subcircuits(circ, cut)
vals = [qd.expectation_value(s) for s in subs]
val = subs.reconstruct(vals)   # auto-parallel
ipycompss.stop(sync=True)
```

**GPU tasks:** a backend explicitly on a GPU (Aer `use_gpu`, CUDA-Q `target='nvidia'`) runs in tasks that ask for one GPU; nothing else is needed. Use GPU nodes: without GPUs those tasks never start. On Apple Silicon add `cpu_affinity='disabled'`.

## OVERHEAD AT A GLANCE

| | |
|---|---|
| `gate cut / cut` | 6 terms, $\gamma = 3$ (optimized: 5 circuits per side). |
| `wire cut / cut` | 8 terms, $\gamma = 4$, 3 if golden (optimized: 3 sent + 4 received; k sent jointly: $2^k+1$). |
| `hadamard / cut` | $\gamma$(CZ)=3, k joint: $2^{k+1}-1$; $\to 1$ for weak gates. |
| `general gate` | up to 3 rzz rotations, each at its angle (one cut with the Hadamard method). |

## SCALE THE PARALLELISM

Every subcircuit is an independent task. Extra knobs for big fan-outs:

| | |
|---|---|
| `sync=False` | Under PyCOMPSs: overlap several cuts, wait once. |
| `circuits_per_task=n` | Fewer, longer tasks for small circuits. |
| `optimized=True` | Gate / wire / mix: a sum over fragments, not a product over cuts. |
| `num_samples=n` | Every cut: draw n configurations, not all of them. |

## GOTCHAS

- Cost is exponential in #cuts ($6^k$ / $8^k$) — fewer, well-placed cuts win; `optimized=True` turns much of it into a sum; preview with `cutting_overhead`.
- Weak gates are cheap to cut. For a gate with several interactions prefer `gate_cutting_hadamard`: Qdislib warns when it is cheaper.
- Sampling needs **more shots** than an expectation value (quasiprobability overhead).
- `error=True` gives a standard error at no device cost; `total_shots=` spends one budget.
- `shots=1024` is the default, on every cut; `shots=None` is exact.
- A cut must separate its two sides, or it raises `InvalidCutError`. Use `seed=` for a reproducible `find_cut`.

## REAL HARDWARE

Route the subcircuits to GPUs or QPUs — the cutting code is unchanged:

```
ibm = qd.build_backend('IBM_BACKEND', qd.BackendConfig(
        extra={'backend_name': 'ibm_marrakesh'}))
qd.gate_cutting(circ, cut, backend=ibm)   # IBM
iqm = qd.build_backend('IQM_BACKEND',
        qd.BackendConfig(quantum_computer='garnet'))
qd.gate_cutting(circ, cut, backend=iqm)   # IQM
qd.gate_cutting(circ, cut,
        backend='BSC_QPU_BACKEND')        # BSC QPU
```

On IBM each subcircuit is one plain job, and the token stays in `$IBM_QUANTUM_TOKEN`. `qd.pack`, or `pack=True` on the IBM backend, runs many subcircuits side by side in one job. The BSC QPUs read `$QDISLIB_SUBCIRCUITS_DIR`.

## BRING YOUR OWN CIRCUIT

Pass a **Qibo, Qiskit, CUDA-Q, PennyLane or QiliSDK** circuit — Qdislib converts it through a neutral DAG that speaks every backend (U3/CU3/… included). Or import OpenQASM 2:

```
circ = qd.from_qasm(open('c.qasm').read(),
        software='qiskit')   # the SDK to build it in
qd.gate_cutting(circ, cut, backend='qiskit')
```

## LEARN MORE

Run it in the browser, or read the docs, the notebooks and the API reference:

| | |
|---|---|
| `Try` | qdislib.readthedocs.io/en/latest/try.html |
| `Explore` | qdislib.readthedocs.io/en/latest/explore.html |
| `Docs` | qdislib.readthedocs.io |
| `Source` | github.com/bsc-wdc/qdislib |
| `Notebooks` | notebooks/, each opens in Colab or Binder. |

## SANITY CHECK

Validate a cut against the exact value on a small instance before you scale it up (cutting reconstructs the same <O>, within shot noise):

```
exact = qd.analytical_solution(circ, 'ZZ')
val   = qd.gate_cutting(circ, cut, 'ZZ',
                        shots=8192)
assert abs(val - exact) < 0.05
qd.draw_cut(circ, cut, 'cut.png')   # see it
```